



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): 6

Integrante(s): 323078401

323115296

323082392

323139441

323253916

No. de lista o

brigada: 04

Semestre: 2027-I

Fecha de entrega: 09/10/2026

Observaciones: _____

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	2
3. Desarrollo	3
3.1. Actividad 1. Análisis de las respuestas de las LLM	3
3.1.1. Análisis de la respuesta 1	3
3.1.2. Análisis de la respuesta 2	3
3.1.3. Análisis de la respuesta 3	4
3.2. Actividad 2. Organización y empaquetamiento de clases	4
4. Resultados	6
5. Conclusiones	7
6. Referencias	7

1. Introducción

Planteamiento del problema

El problema consiste en establecer una organización uniforme para clasificar y conservar todos los trabajos realizados en el semestre dentro de un paquete general.

Motivación

Almacenar los archivos con código fuente en un espacio propio y aislado de los archivos binarios compilados para que los archivos editables no se mezclen con los productos generados por java. Asimismo, agrupar las aplicaciones empaquetadas en un directorio específico y reunir la documentación en una carpeta común.

Objetivos

- Crear un paquete general para organizar las prácticas y proyectos acumulados del curso.
- Distribuir de manera diferente el código fuente, los archivos binarios compilados, los JAR y la documentación de cada trabajo.
- Generar los JAR y la documentación correspondiente a partir de los archivos del proyecto utilizando las herramientas del entorno de Java.

2. Marco Teórico

Los paquetes cumplen la función de evitar conflictos de nombres entre clases desarrolladas por diferentes programadores y permiten gestionar los niveles de visibilidad y acceso a los miembros de cada componente [1]. La pertenencia de una clase a un paquete se establece al inicio del archivo fuente. Debe existir una correspondencia directa entre el nombre del paquete y la estructura de directorios en el sistema de archivos[1].

Un archivo JAR (Java ARchive) es un formato de archivo comprimido basado en el estándar ZIP que se utiliza para empaquetar múltiples archivos compilados de Java junto con sus metadatos y recursos asociados en un único elemento distribuible [2]. Cuando se define en este manifiesto la cabecera correspondiente a la clase principal de la aplicación, el archivo JAR se convierte en un paquete ejecutable directamente por la máquina virtual de Java, lo que permite iniciar el programa sin requerir la especificación manual de la clase con el punto de entrada desde la consola [2].

Por su parte, Javadoc es la herramienta estándar incluida en el kit de desarrollo de Java (JDK) encargada de generar documentación técnica estructurada en formato

HTML a partir del código fuente y de comentarios especiales delimitados en el texto [3]. Esta herramienta analiza las declaraciones de clases, métodos y campos públicos y protegidos, así como etiquetas formales destinadas a especificar parámetros, valores de retorno, versiones y excepciones [3].

3. Desarrollo

3.1 Actividad 1. Análisis de las respuestas de las LLM

En esta actividad se consultó a diferentes modelos de lenguaje sobre conceptos de programación orientada a objetos y buenas prácticas de desarrollo en Java, específicamente en relación con el uso de archivos comprimidos JAR y la generación de documentación mediante Javadoc.

3.1.1 Análisis de la respuesta 1

En la primera consulta, el integrante solicitó buenas prácticas para el uso de JAR y Javadoc, solicitando una justificación técnica, los beneficios asociados y una explicación argumentada sobre la mejor manera de generar la documentación.

Frente a esto, el modelo estructuró una respuesta orientada a estándares profesionales avanzados. Sugirió emplear herramientas de construcción automatizadas como Maven y Gradle para generar tanto los archivos JAR como el Javadoc, desaconsejando el uso directo de las herramientas de consola en proyectos formales para asegurar la reproducibilidad del proceso. Asimismo, detalló buenas prácticas de documentación enfocadas en describir contratos, condiciones de error y versiones.

La respuesta fue técnicamente rigurosa y dio respuesta a los puntos solicitados. El modelo asumió un entorno corporativo con procesos de integración continua y modularidad, omitiendo el procedimiento de consola.

3.1.2 Análisis de la respuesta 2

En la segunda consulta, la solicitud se planteó con un lenguaje informal y directo, preguntando por las buenas prácticas de JAR y Javadoc y pidiendo argumentar cuál es la mejor forma de generar la documentación. El modelo interpretó de manera natural un entorno de aprendizaje universitario.

Su respuesta se centró en distinguir con claridad que los archivos JAR se utilizan para la distribución y empaquetado del programa compilado, mientras que Javadoc

tiene como único fin generar documentación técnica para la API. Como recomendación principal, el modelo propuso organizar el proyecto mediante carpetas diferenciadas para fuentes, binarios, documentación y distribución, y sugirió generar el Javadoc directamente mediante la herramienta de consola del entorno de Java.

La solución abordó todas las preguntas del prompt, explicando las etiquetas de documentación más comunes y el proceso para empaquetar ejecutables mediante consola. Este enfoque didáctico y contextualizado permitió al modelo enfocar su respuesta en el flujo de trabajo básico de un estudiante.

3.1.3 Análisis de la respuesta 3

En la tercera consulta, el integrante formuló una petición extensa y altamente detallada, requiriendo buenas prácticas, fuentes bibliográficas fidedignas para respaldar qué prácticas son adecuadas o inadecuadas, y restringiendo la explicación al contexto del tema de herencia en programación orientada a objetos. Sin embargo, en el texto de la consulta se escribió por error “Java y JavaDoc” en lugar de solicitar información sobre archivos JAR.

Como consecuencia de esta indicación estricta y del desliz en la redacción, el modelo generó una respuesta académica profunda y fundamentada mediante literatura especializada sobre diseño orientado a objetos y contratos de extensión en clases derivadas, citando obras formales de ingeniería de software. No obstante, omitió por completo el tema de los archivos JAR debido a que la palabra no formaba parte de la solicitud.

Este prompt con restricciones rígidas obligó al modelo a ceñirse exclusivamente a los términos explícitos, dejando de lado los aspectos que se pretendían consultar pero que no se redactó de forma precisa.

La comparación entre las tres consultas muestra que las respuestas de los modelos dependen en gran medida del contexto y la precisión de las instrucciones proporcionadas. Cuando la consulta carece de límites, el modelo tiende a responder con estándares profesionales complejos; cuando se plantea de forma abierta y sencilla, ofrece soluciones pedagógicas orientadas a la consola; y cuando se especifican restricciones conceptuales estrictas, el modelo se enfoca en ellas con rigor, reflejando de forma inmediata cualquier omisión o error presente en la redacción del usuario.

3.2 Actividad 2. Organización y empaquetamiento de clases

El trabajo principal de la práctica consistió en organizar las clases y los componentes de las prácticas y proyectos realizados a lo largo del semestre dentro de un

paquete general, asegurando la separación física de los distintos tipos de archivos y generando tanto los JAR así como la documentación técnica correspondiente.

La estructura general se organizó a partir de una carpeta principal de almacenamiento que reúne seis divisiones independientes, correspondientes a cinco prácticas de laboratorio y un proyecto. Cada una de estas divisiones contiene internamente un esquema de cuatro carpetas: una carpeta denominada `src` para el código fuente, una carpeta denominada `test` para los archivos binarios compilados, una carpeta denominada `library` para los JAR y una carpeta denominada `document` para los reportes y la documentación técnica generada.

En este diseño resulta fundamental distinguir las carpetas de los paquetes de Java. Los paquetes de Java declarados formalmente en los archivos fuente reproducen la ruta que ubica la pertenencia de las clases dentro de la universidad, la facultad, la división académica, la materia, el grupo y la práctica específica. Esta ruta de paquetes se encuentra contenida exclusivamente dentro de las carpetas `src` y `test`, garantizando que la estructura física coincida de forma exacta con la declaración formal del paquete en cada una de las dieciséis clases que integran los trabajos.

La separación de los directorios permite que cada tipo de archivo cumpla una función bien definida dentro del conjunto. La carpeta `src` almacena únicamente los archivos con extensión `.java`, que representan el código fuente editable mantenido por el equipo. La carpeta `test`, almacena exclusivamente los archivos con extensión `.class` generados tras el proceso de compilación, conservando exactamente la misma estructura de paquetes que la carpeta de fuentes. Por su parte, la carpeta `library` concentra las aplicaciones distribuidas en formato JAR. Finalmente, la carpeta `document` agrupa tanto el reporte formal de cada práctica en formato PDF como una subcarpeta con los archivos HTML producidos por la herramienta de documentación.

Para la generación de los distintos elementos se utilizaron las herramientas provistas por el entorno de desarrollo de Java. En primer lugar, la compilación de los archivos fuente se realizó mediante el compilador de Java. A través de las opciones de compilación, se tomó como entrada el código ubicado en la carpeta de fuentes y se indicó como destino la carpeta `test`, lo que permitió que el compilador creara automáticamente la jerarquía de subcarpetas requerida por los paquetes y depositara en ella los archivos binarios `.class`.

A partir de los archivos compilados presentes en la carpeta de binarios, se empleó la herramienta de creación de archivos JAR para empaquetar las clases de cada práctica en un solo archivo comprimido depositado dentro del directorio `library`. Durante este procedimiento, se configuró el empaquetador para generar un archivo detallando su nombre de salida, agregando los archivos compilados desde el directorio actual e

indicando cuál de las clases del paquete contenía el método principal. Esto permitió que el manifiesto interno del paquete JAR quedara registrado con la clase de arranque correspondiente, convirtiendo al archivo en un elemento ejecutable de manera directa mediante la máquina virtual sin requerir la intervención de bibliotecas externas. La verificación del funcionamiento de este empaquetado se realizó ejecutando la aplicación directamente desde el archivo generado, obteniéndose en pantalla el comportamiento y la salida habitual de la clase principal correspondiente a la práctica.

Finalmente, la documentación se generó a partir de los archivos fuente almacenados en la carpeta src mediante la herramienta Javadoc. En el procedimiento se especificó la ruta de destino dentro de la carpeta document, se indicó la raíz de búsqueda del código fuente y se procesó el paquete completo de la práctica. Asimismo, se configuró la herramienta para manejar la codificación de caracteres en formato UTF-8 y para incluir en las páginas resultantes la información de autoría, versiones de las clases y elementos con visibilidad de paquete. Como resultado, la herramienta generó un conjunto estructurado de páginas HTML, índices y hojas de estilo que describen las clases y métodos documentados de la práctica.

A través de esta estructura, los archivos fuente permanecen protegidos e independientes, los productos compilados y empaquetados pueden reconstruirse o distribuirse sin afectar el código base, y la documentación queda disponible para su consulta.

4. Resultados

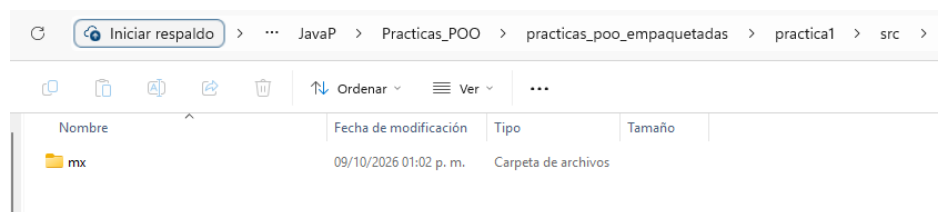


Figura 1: Estructura general de directorios mostrando las carpetas y los paquetes de Java

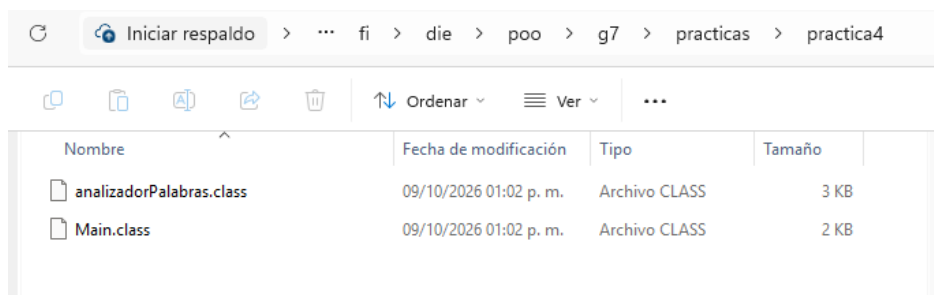


Figura 2: Contenido de la carpeta de archivos compilados con los archivos binarios generados

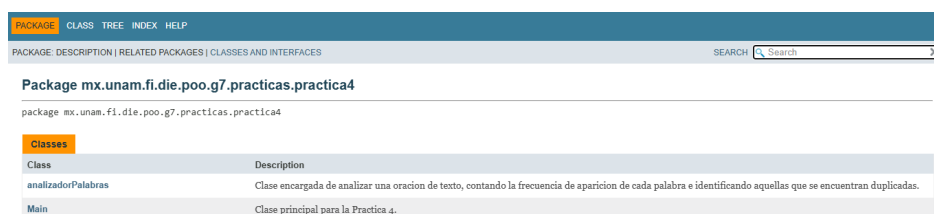


Figura 3: Documentación técnica generada en formato web visualizada en el navegador

5. Conclusiones

Los resultados obtenidos permiten observar que la conformación de una estructura de directorios resuelve de manera efectiva el desorden derivado de la acumulación de archivos. Asimismo, la aplicación de los conceptos teóricos relacionados con los paquetes de Java y las herramientas de empaquetado y documentación resultó necesario para lograr un paquete funcional. Por otra parte, la generación de archivos JAR con su clase principal definida en el manifiesto mostró una gran utilidad en agrupar los componentes compilados en un formato transportable y autónomo, mientras que la producción de documentación mediante Javadoc evidenció que los contratos e interfaces de las clases pueden comunicarse formalmente a partir del propio código.

6. Referencias

[1] Oracle Corporation, “Creating and Using Packages,” *The Java™ Tutorials*, Oracle Documentation. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial>

[ial/java/package/packages.html](http://docs.oracle.com/javase/7/docs/tutorial/java/package/packages.html)

[2] Oracle Corporation, “Packaging Programs in JAR Files,” *The Java™ Tutorials*, Oracle Documentation. [En línea]. Disponible en:<https://docs.oracle.com/javase/tutorial/deployment/jar/>

[3] Oracle Corporation, “How to Write Doc Comments for the Javadoc Tool,” *Oracle Technical Articles*, Oracle Documentation. [En línea]. Disponible en:<https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>